

INFOMCV Assignment 2

Majo Bednár (2439905), Angel Bu Tong Mei (9682120) (Group 98)

Summary

We calibrate each camera by selecting a number of frames (around 60, depending on video) spaced across the intrinsics.avi videos (also see choice task 7). We then use the intrinsics and the first frame of checkerboard.avi to calculate the extrinsics.

After we have determined the camera parameters, we process the videos (that we use for voxel reconstruction) by doing the background subtraction of every frame in the video, and thus create a new background/foreground video, for each camera. See next section for details of background subtraction.

To build the voxel model, we start by choosing linearly spaced world coordinates that capture the person in the space. We decided to space the points 20 millimetres apart. We first create a lookup table from these world coordinates – voxels. Each voxel key maps to each camera, from which the projection to 2D is stored. Then, we build the voxel model by iterating over the voxels. For each voxel, and each camera, we check if all 2D projections are in the picture bounds and if all correspond to foreground. If so, we add this voxel to the list of visible voxels. We do this procedure for multiple different frames of the four videos corresponding to every camera. We use the videos that are processed by the background subtraction, thus having separate background (black) and foreground (white) already.

Extrinsic parameters

Table below shows the rotation and translation vectors for each camera:

Camera 1		Camera 2		Camera 3		Camera 4	
Rotation vector	Translation vector	Rotation vector	Translation vector	Rotation vector	Translation vector	Rotation vector	Translation vector
[-0.67 -1.62 -1.89]	[410.66 915.96 3577.61]	[-0.05 -2.22 -2.17]	[546.16 1497.27 3002.96]	[-1.26 1.02 1.04]	[-349.11 1199.55 3349.20]	[-0.75 1.58 1.79]	[-20.02 896.70 4267.75]

Background subtraction

Background Model: We build a background model for each camera by sampling every third frame of background.avi, converting them to HSV colour space, and computing the median for each pixel.

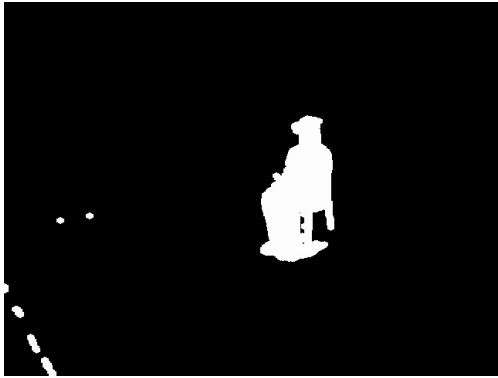
Foreground Classification: Frames from video.avi are converted to HSV and compared pixel-by-pixel against the background model. We compute the absolute difference for Saturation (S) and Value (V), and the circular difference for Hue (H) to account for its 180° wrap-around.

Thresholds: A pixel is classified as foreground (255) if the difference in any channel exceeds its threshold; otherwise, it is background (0). Thresholds are either fixed empirically (H=10, S=40, V=40) or automatically optimized by minimizing noise (small connected components), see choice tasks.

Post-Processing: To clean the raw binary masks, we apply morphological erosion followed by dilation using a 5×5 elliptical kernel, effectively removing speckle noise while preserving object boundaries and filling holes.

Results: Representative foreground masks for all four cameras are shown below.

Cam 1



Cam 2



Cam 3



Cam 4



Choice tasks

We did choice tasks 2, 3, 4 and 7.

Choice task 2: Thresholds are chosen automatically by minimizing noise. For each candidate (H, S, V) combination, we compute the foreground mask, apply erosion/dilation, and count small connected components (area < 50–100 pixels). The combination with the fewest such blobs is selected. This is done on the first frame per camera, without manual input. The candidates are as follows:

Hue: 5, 10, 15, 20

Saturation: 20, 30, 40, 50

Value: 20, 30, 40, 50

Choice task 3: We first determine for each camera which voxels are visible. We do this by computing the depth (distance) to the voxel. For each pixel projection we save the smallest depth and the corresponding (closest) voxel. Then for each voxel we compute its colour by taking the median from the four cameras (if a voxel is not visible from a camera the colour from this view is simply not considered). If a voxel is not visible from any camera, we assign a black colour.

Choice task 4: We convert the voxel reconstruction output into a surface mesh using Marching Cubes. First, we build a binary 3D volume from the voxel list: we shift coordinates, so they are non-negative, discretize by voxel size, and fill a 3D array with 1s where voxels exist. We then run Marching Cubes (from scikit-image) on this volume with level=0.5 to get vertices and triangle faces at the surface. The vertices are scaled and shifted back to world coordinates. The mesh is visualized with Open3D.

Choice task 7: we added an automatic frame-quality filter that runs after chessboard detection (different to the low-quality image detection we implemented in assignment 1). It rejects frames that are too blurry (Laplacian variance below 100) or have irregular corner spacing (coefficient of variation above 0.35). Only frames that pass these checks are used for calibration.

Link to video

https://drive.google.com/file/d/1JJ6tzghADB0OjVrBy70pNv_MSfJM-56O/view?usp=sharing

Any genAI prompt used

(Chatgpt) What is a good way to visualize mesh in python, given by scikit-image marching cubes algorithm. (Many ways recommended, we chose open3d)

(Chatgpt) What color should I assign if a voxel is not visible by any camera? (again many options recommended, we just went for black)

(Chatgpt) What is a good structure for lookup table for 3D voxel reconstruction? (recommended many options, the output was quite used in the end)

(Chatgpt) Is reading video frames in python opencv with a while loop reasonable? (essentially the answer was yes if not too long)

(Chatgpt) How to save/read from an xml file in python? (Used for saving/reading configuration)

(Chatgpt) How to save dictionary of dictionaries of numpy arrays? (for storing the lookup table, we used .npz)